

3. Mobile Application Models

Objectives

Explain the concept of an Activity

Explain the Intent and Intent Filter

Explain the lifecycle of an Activity

Manage the Activity lifecycle

Activity

Activity = UI

The “main” activity is the first UI

Activity can be started by another activity within the same app or other apps

Activity

It must be declared in the Manifest File in order for it to be accessible to the system

```
<manifest ... >
  <application ... >
    <activity android:name=".ExampleActivity" />
    <activity android:name=".SecondExampleActivity" />
    ...
  </application ... >
  ...
</manifest >
```

Activity

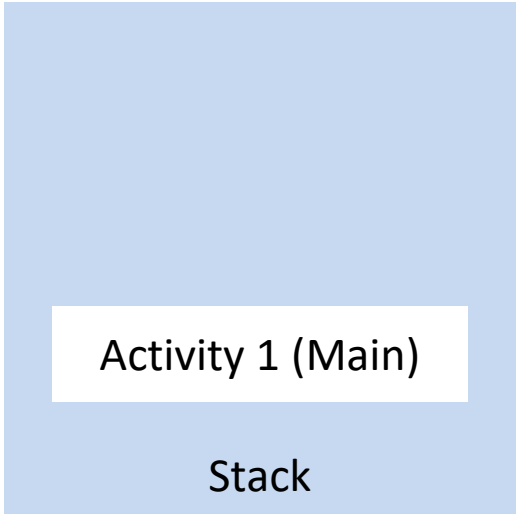
To shut down an Activity:

- [finish\(\)](#) method: shutdown current activity
- [finishActivity\(\)](#) method: shutdown a separate activity that was previously started

Activity

Each time a new activity starts, the previous activity is stopped, but the system preserves the activity in a stack (last in, first out)

User press the *Back* button, the current activity is popped from the stack and the previous activity resumes

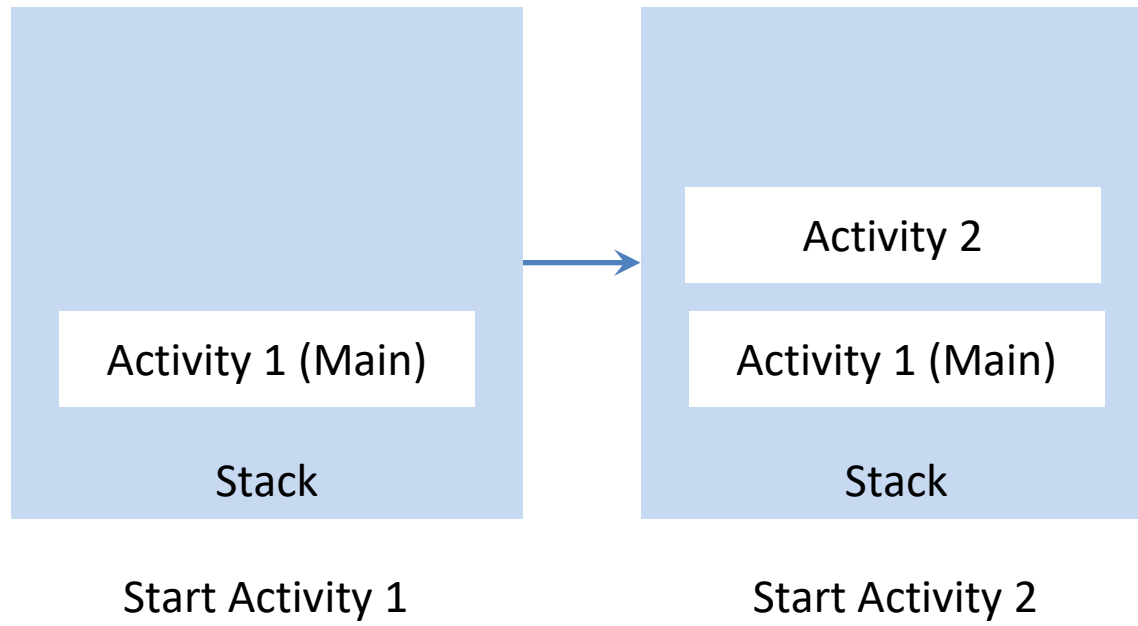


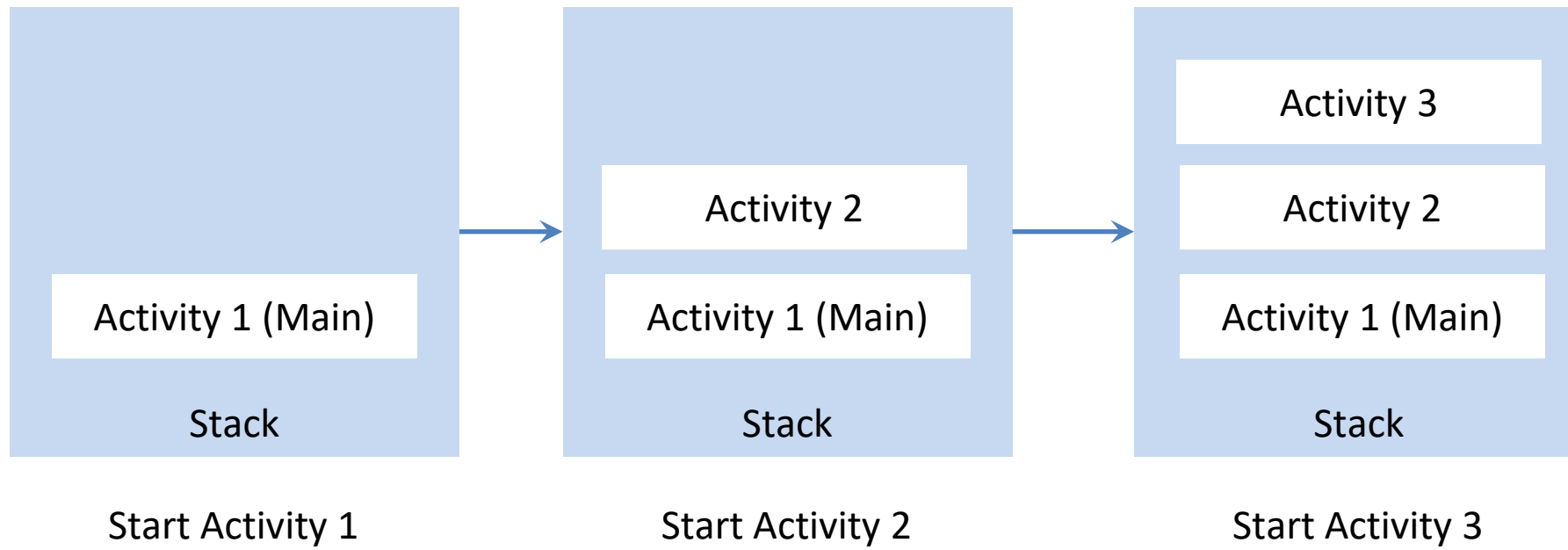
Activity 1 (Main)

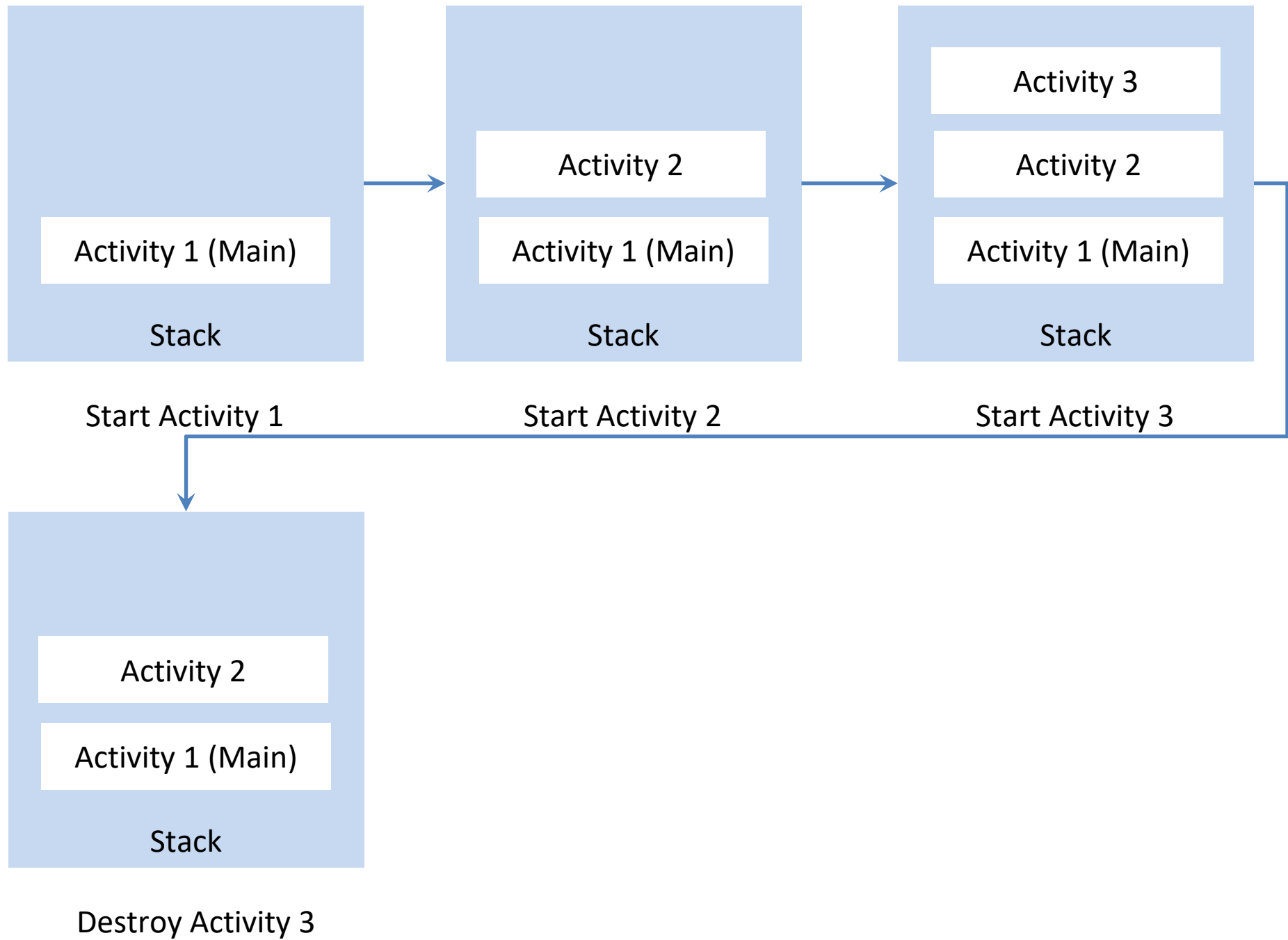
The diagram shows a light blue rectangular container representing a stack. Inside the container, at the bottom, is a white rectangular box labeled "Activity 1 (Main)". Below the container, the text "Start Activity 1" is written.

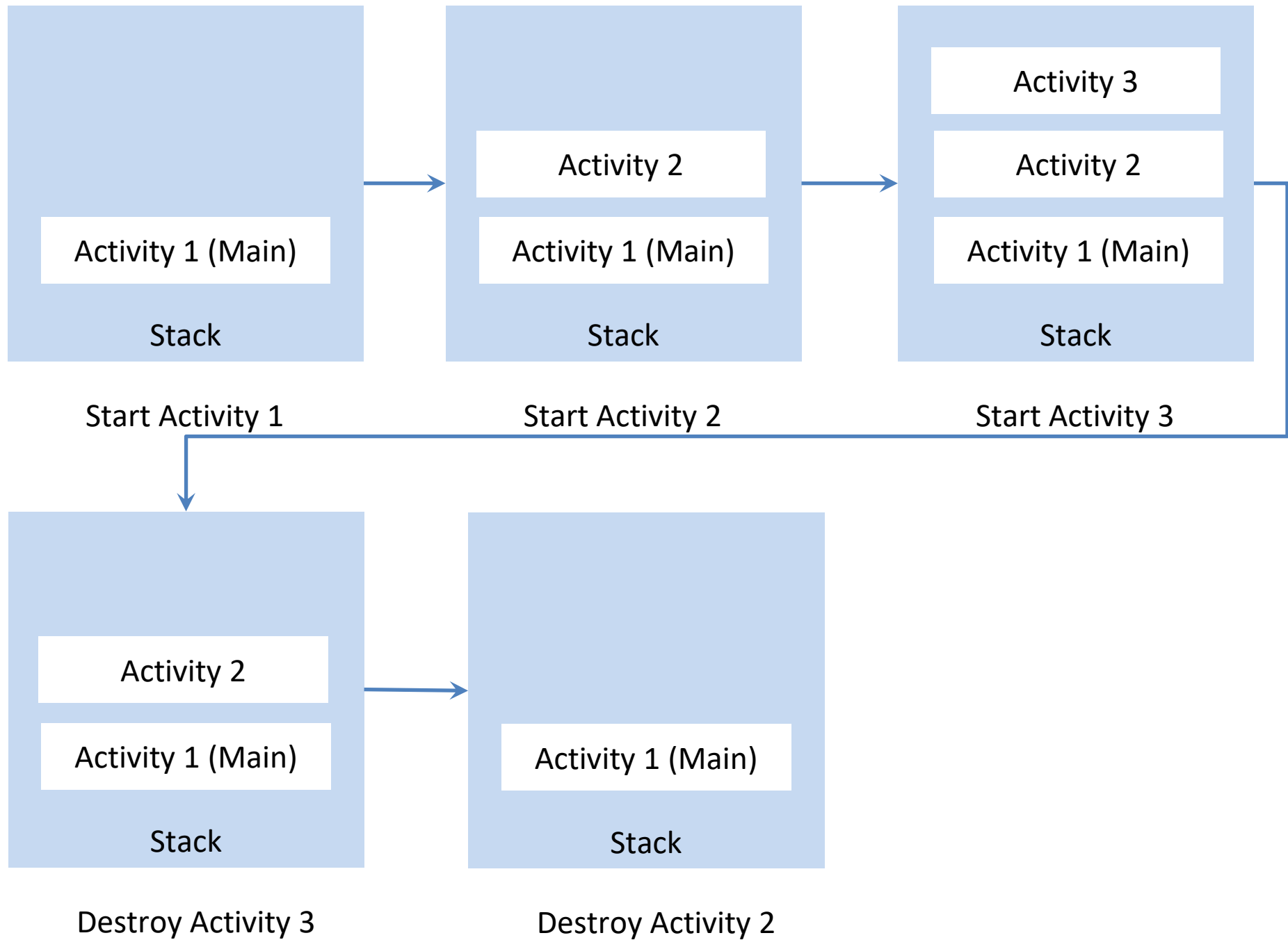
Stack

Start Activity 1



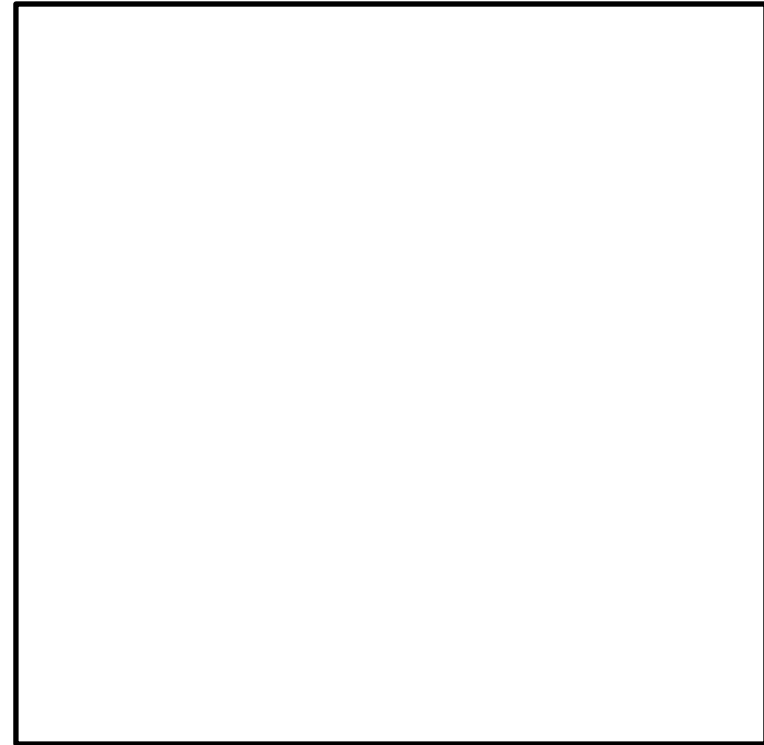






Question?

1. In the context of Android, what is the main purpose of an Activity?
2. Why it is important to declare an Activity in the manifest file?
3. What is the data structure used by Android to manage components of an app?
4. What will happen to an Activity if a user navigates away from it?



Intent

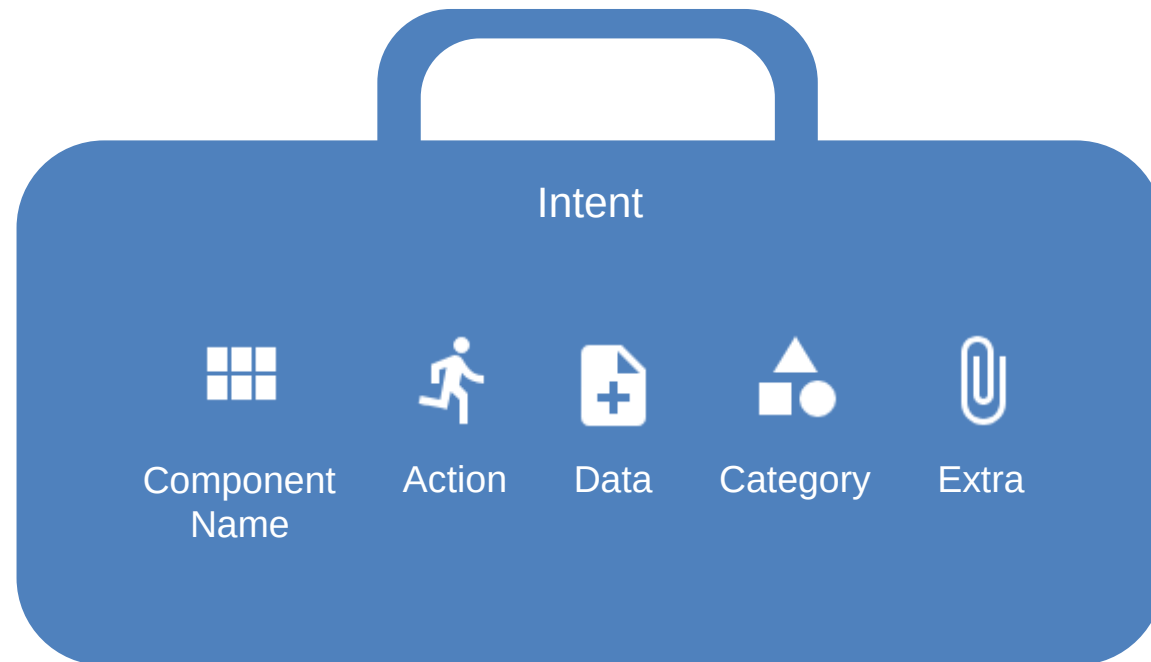
Intent is used to start an Activity

Two types of Intent:

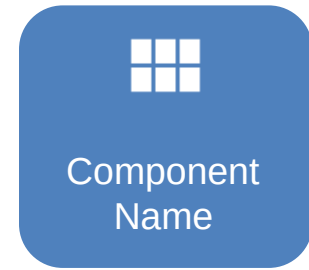
Type	Description
Explicit	To start a component in your own app. You need to supply a target app's package name or a fully-qualified component class name
Implicit	Declare a general action to perform, which allows a component from another app to handle it

Intent

An Intent carries the following information that the Android System uses to perform the action:



Intent - Component



The exact name of the component to start

Use the Explicit Intent to launch another Activity within an app

When starting a Service, always specify the component name for better security

Intent - Action



It determines how the rest of the Intent is structured – data and extras

Common actions are : `ACTION_VIEW` and `ACTION_SEND`

Intent - Data



The Uniform Resource Identifier (URI) that references the data to be acted on and/or the Multipurpose Internet Mail Extension (MIME) type of that data

Action	Data	Type
ACTION_VIEW	content://contacts/people/1	
ACTION_VIEW	tel:031234567	
ACTION_VIEW	geo:47.6, -122.3	
ACTION_VIEW	http:www.example.com	
ACTION_SEND	This is my text to send	text/plain
ACTION_SEND	(URI to image)	image/jpeg

Intent - Category



Optional

Additional information about the kind of component that should handle the intent

Common categories : CATEGORY_BROWSABLE and CATEGORY_LAUNCHER

Intent - Extra

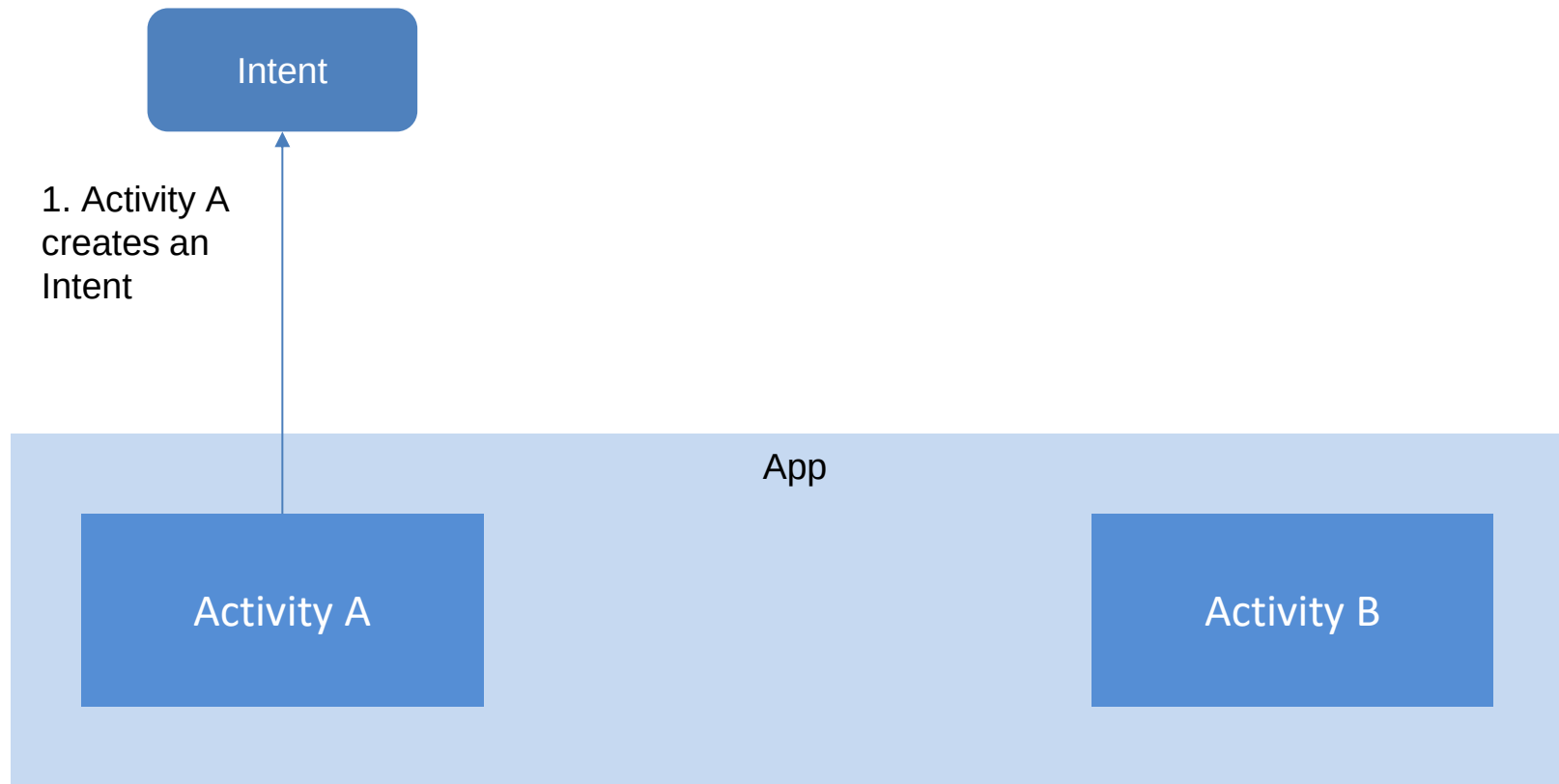


Key-value pairs that carry additional information required to accomplish the requested action

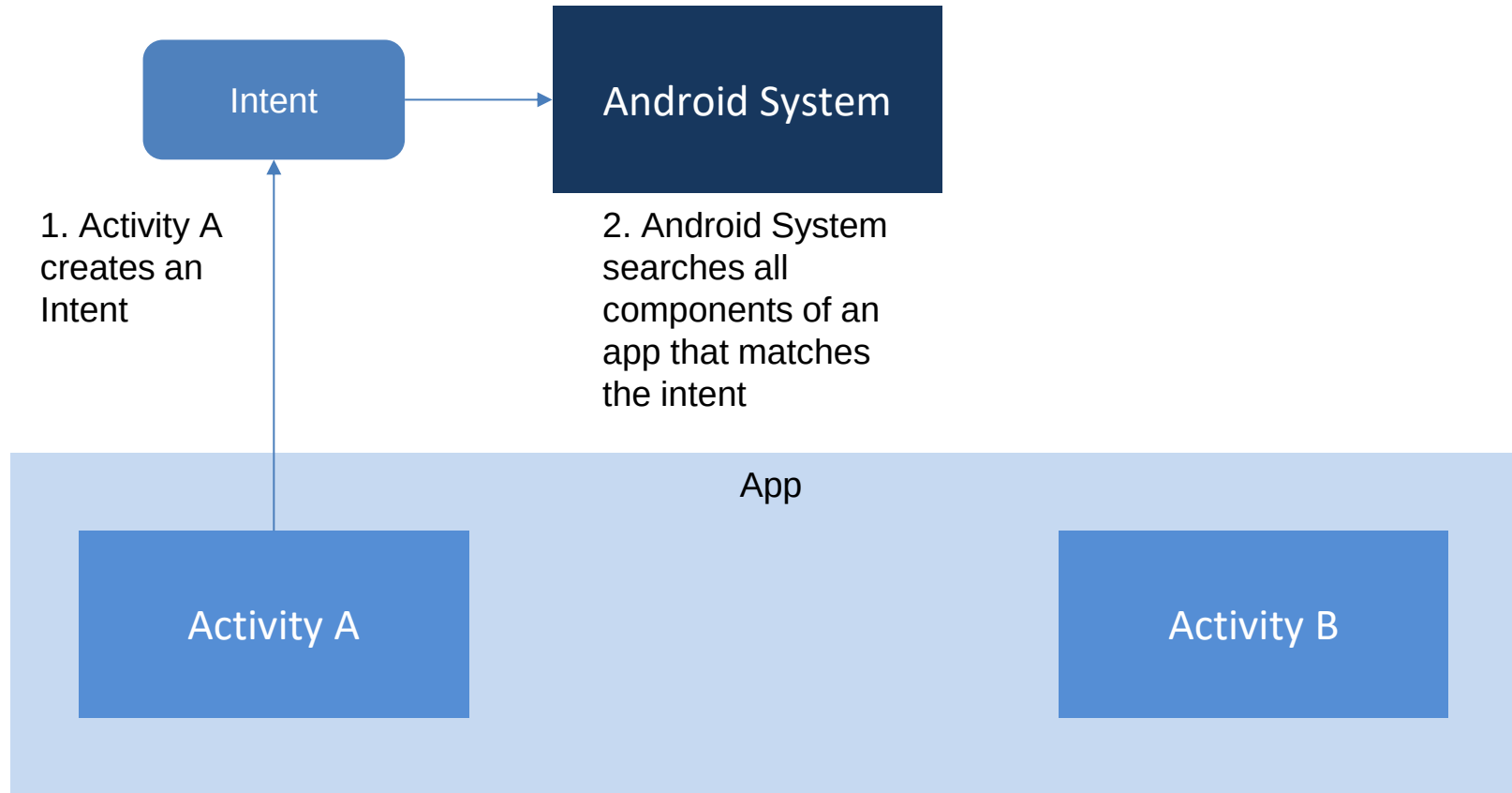
Use the `putExtra(Key, Value)` method to insert data to an Intent

Use the `getExtras()` to retrieve data from an Intent

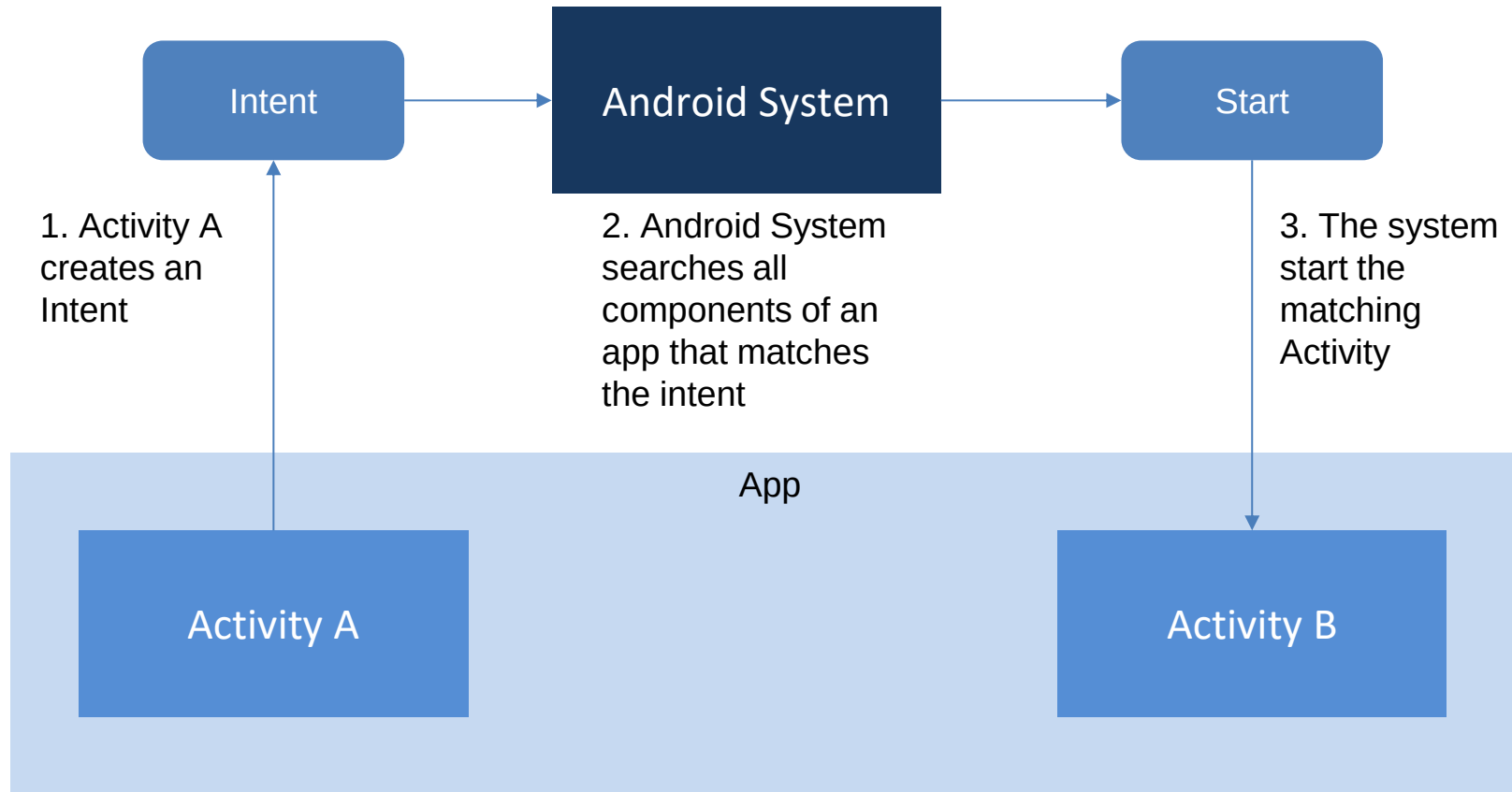
Intent - Explicit



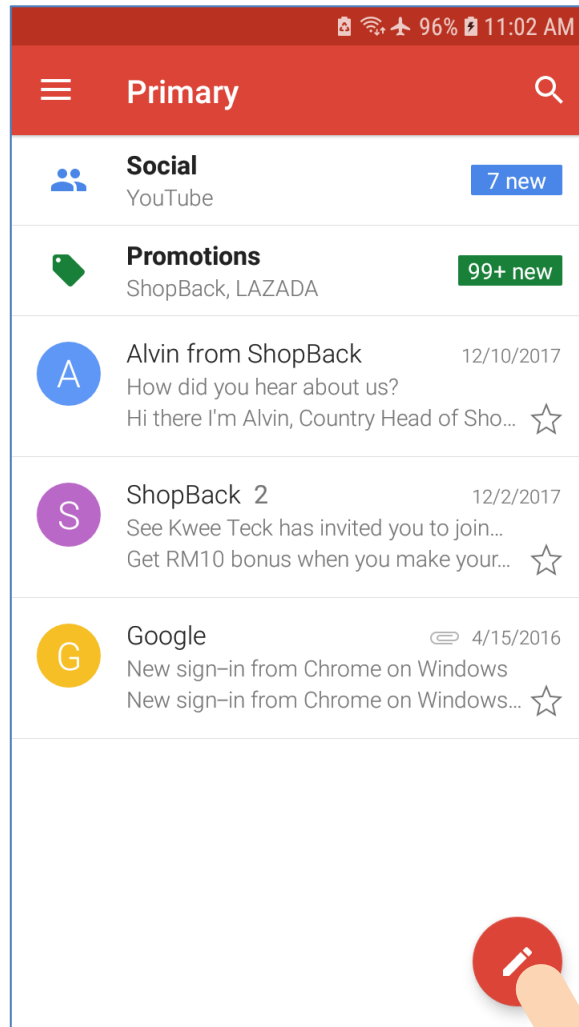
Intent - Explicit



Intent - Explicit

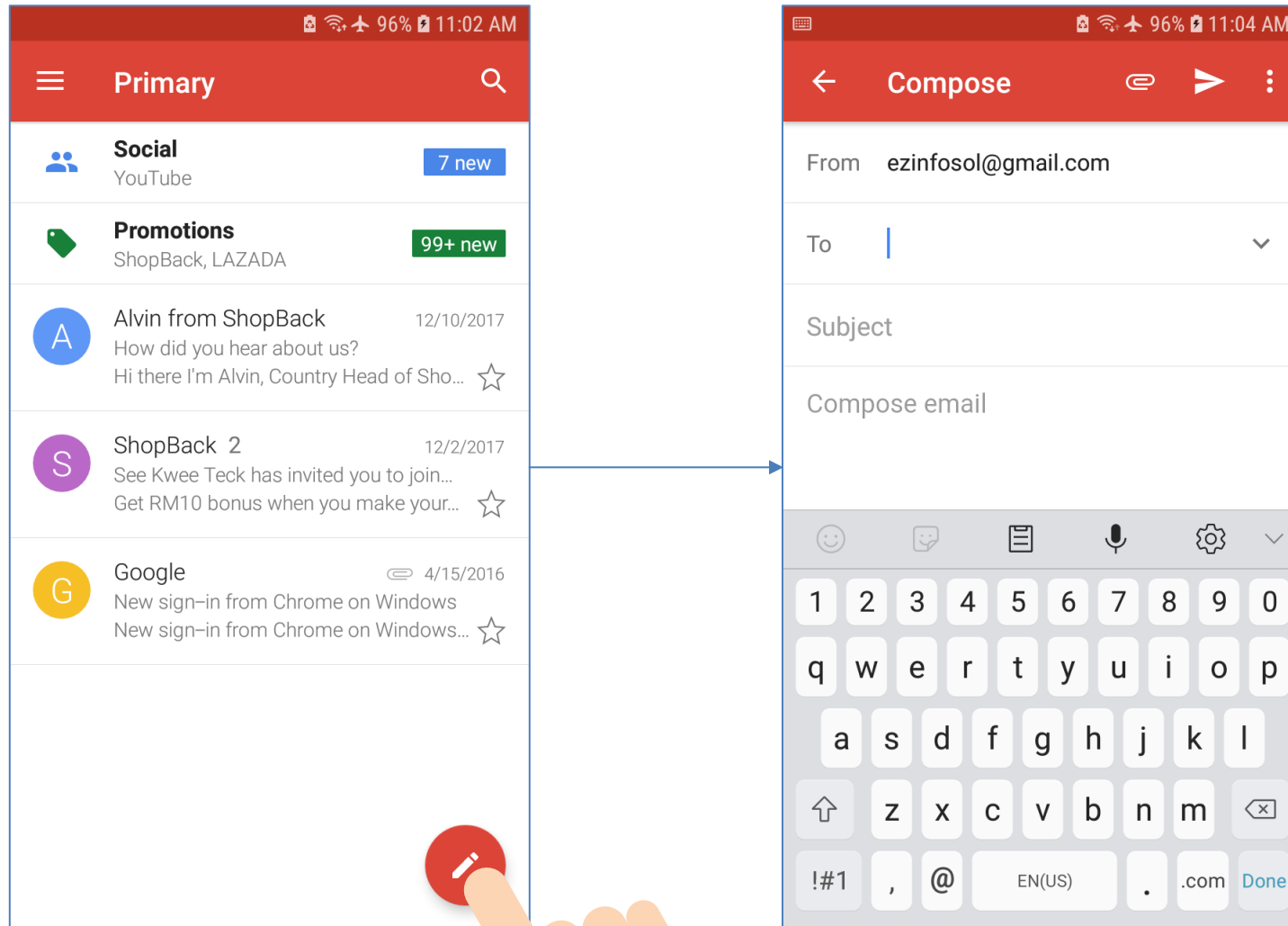


Intent - Explicit



Gmail (Main)

Intent - Explicit



Gmail (Main)

Gmail (Compose)

Intent - Explicit

Start another activity by calling [startActivity\(\)](#), passing it an [Intent](#) that describes the activity you want to start.

Codes in the Gmail main Activity:

```
intent = Intent(context, ComposeActivity::class.java)
```

```
startActivity(intent)
```

Intent - Explicit

An Intent carries data from one Activity to another using the Extra

```
const val EXTRA_MESSAGE = "com.example.myfirstapp.MESSAGE"
const val EXTRA_VALUE = "com.example.myfirstapp.VALUE"
...
val intent = Intent(this, DisplayMessageActivity::class.java).apply {
    putExtra(EXTRA_MESSAGE, message)
}
```

OR

```
intent.putExtra(EXTRA_MESSAGE, message)
```

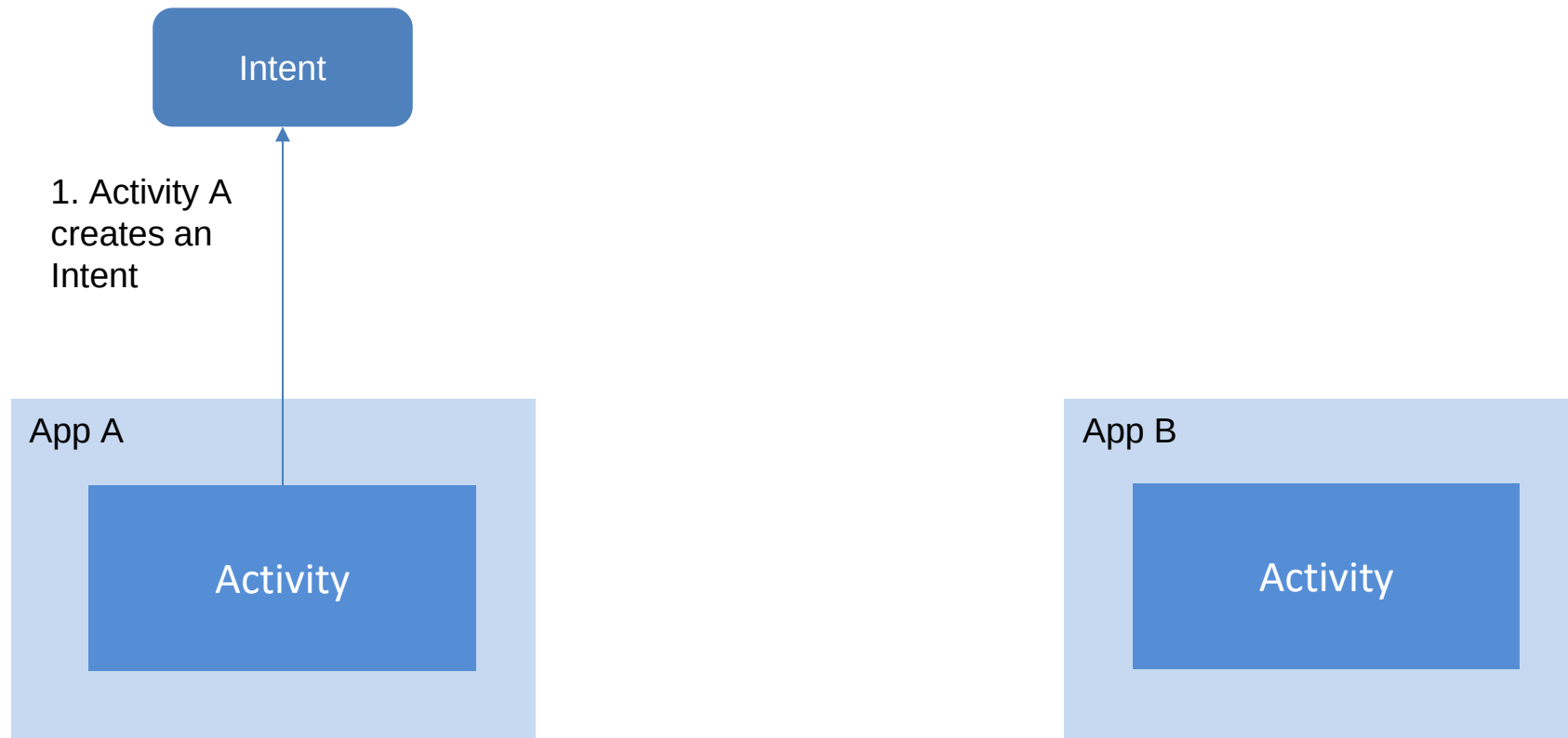
```
startActivity(intent)
```

Intent - Explicit

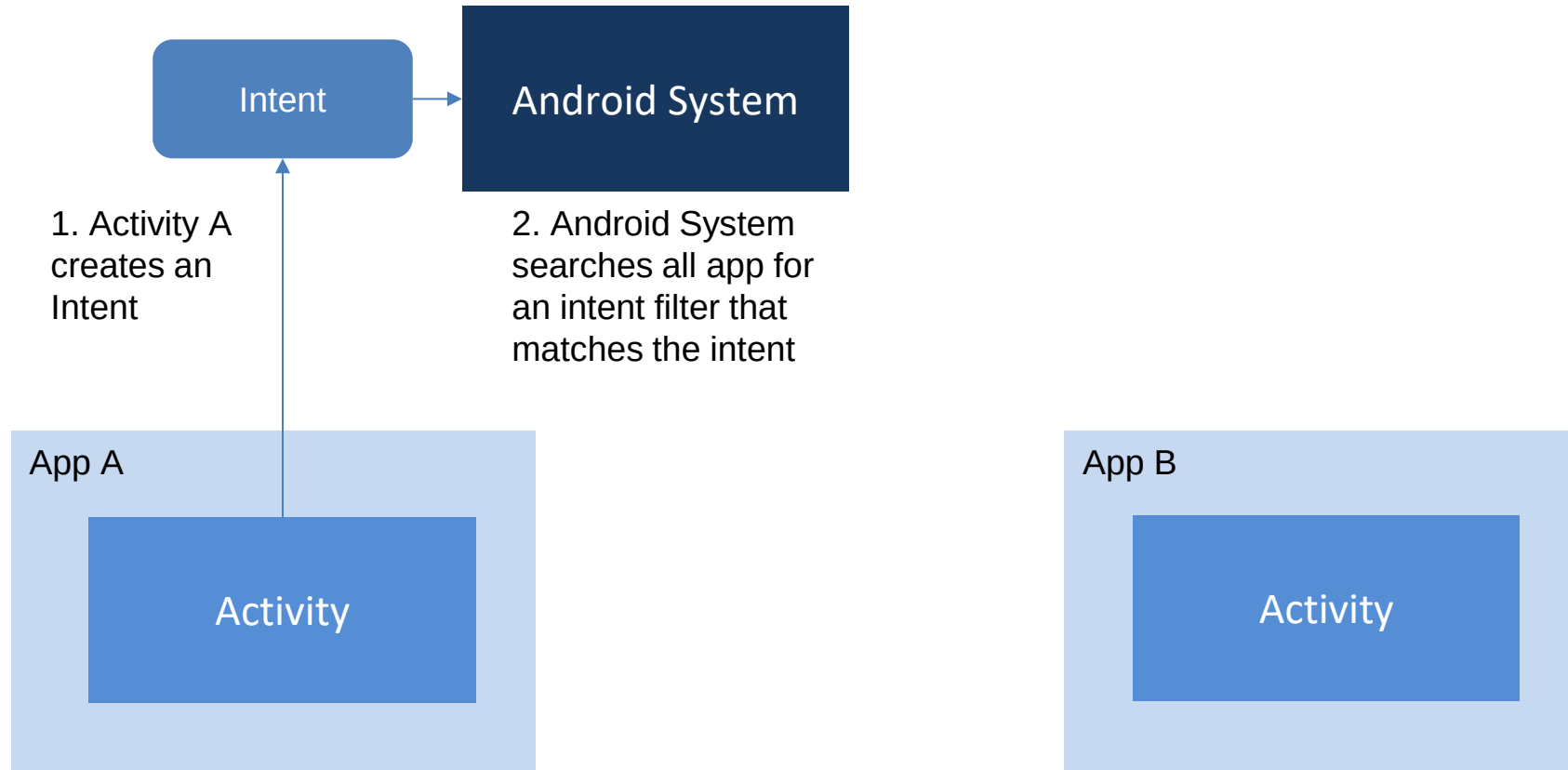
To receive data from an Activity, use the Intent's get methods

```
// Get the Intent that started this activity and extract the string  
val message = intent.getStringExtra(EXTRA_MESSAGE)
```

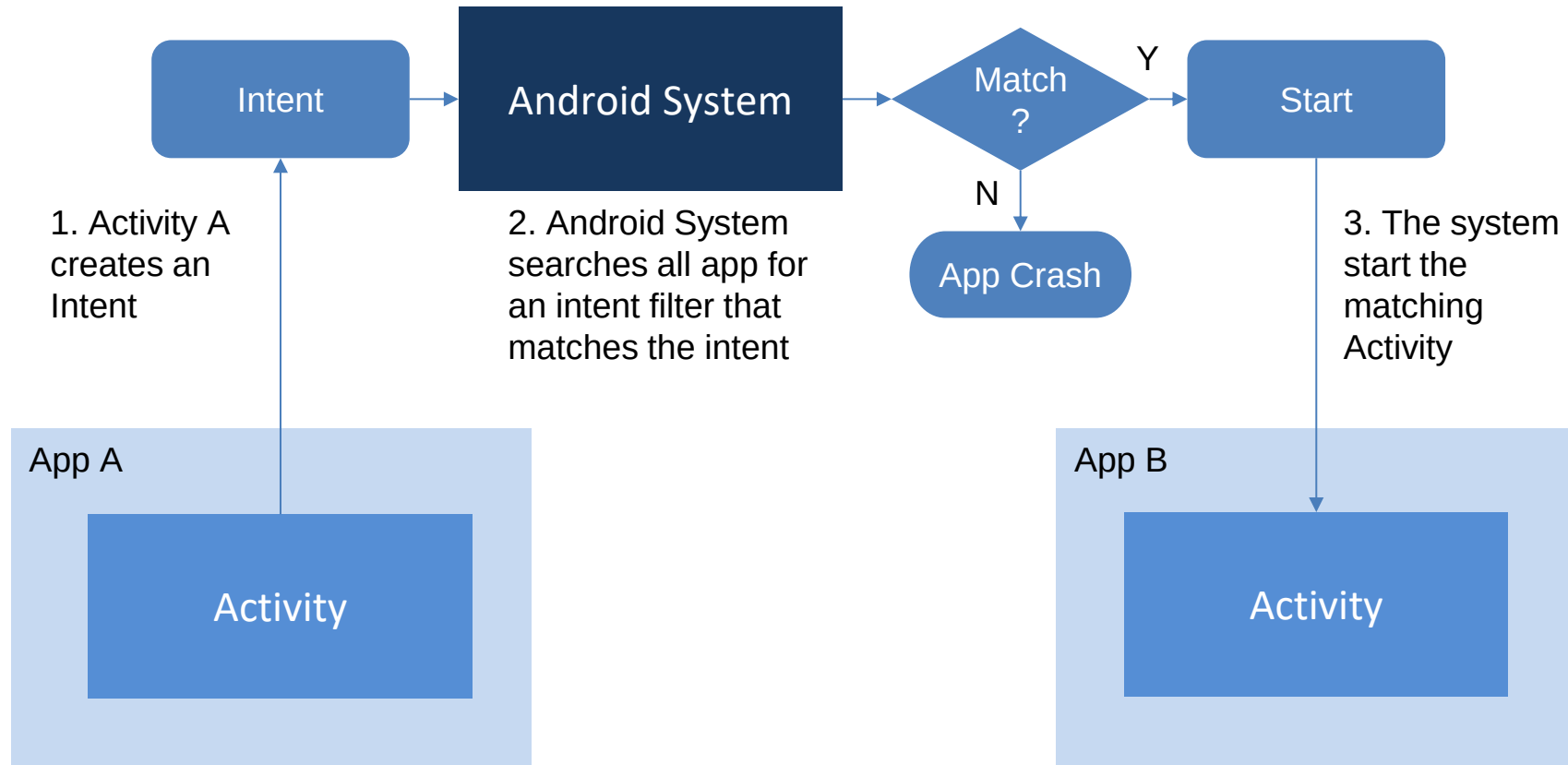
Intent - Implicit



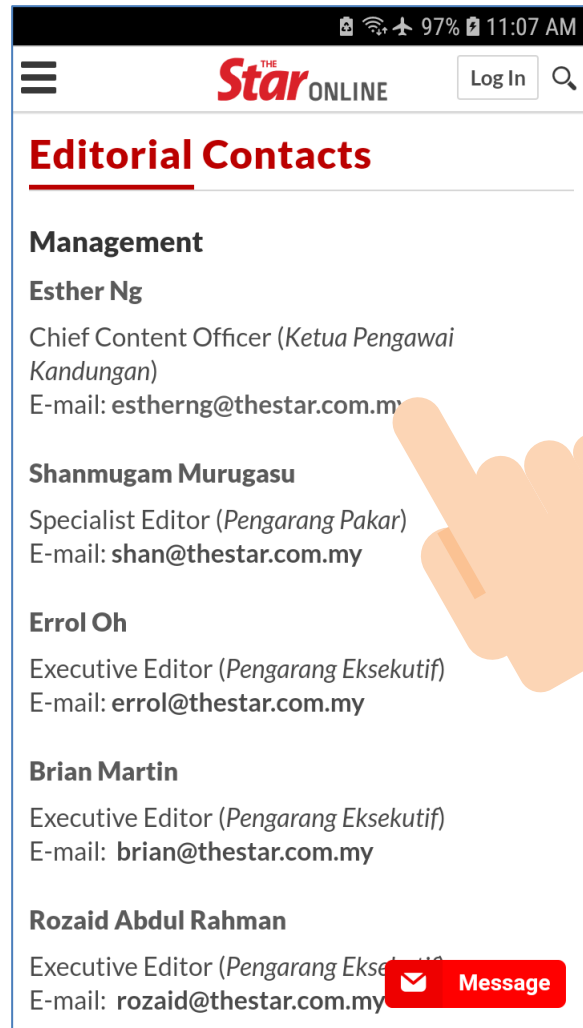
Intent - Implicit



Intent - Implicit

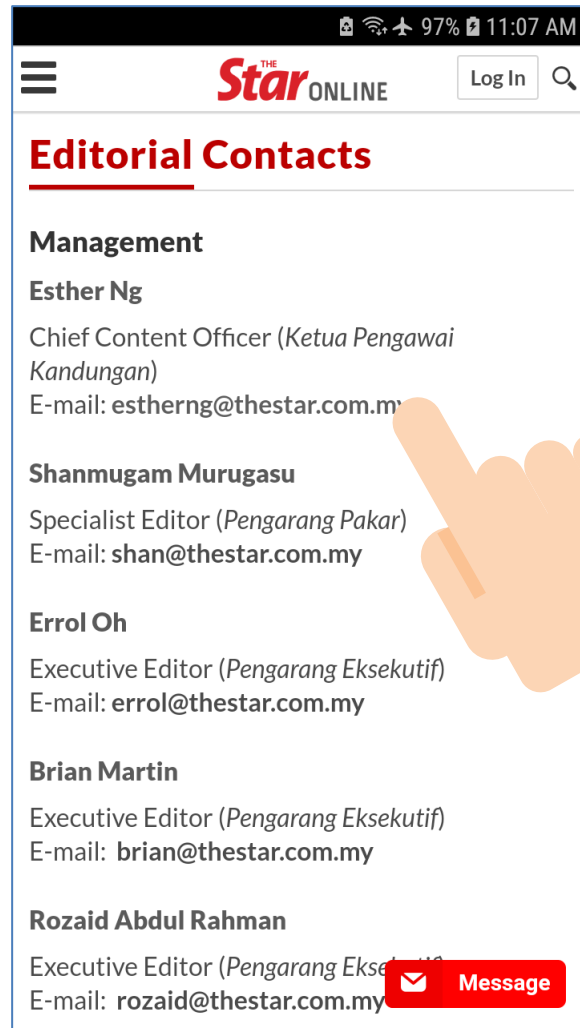


Intent - Implicit

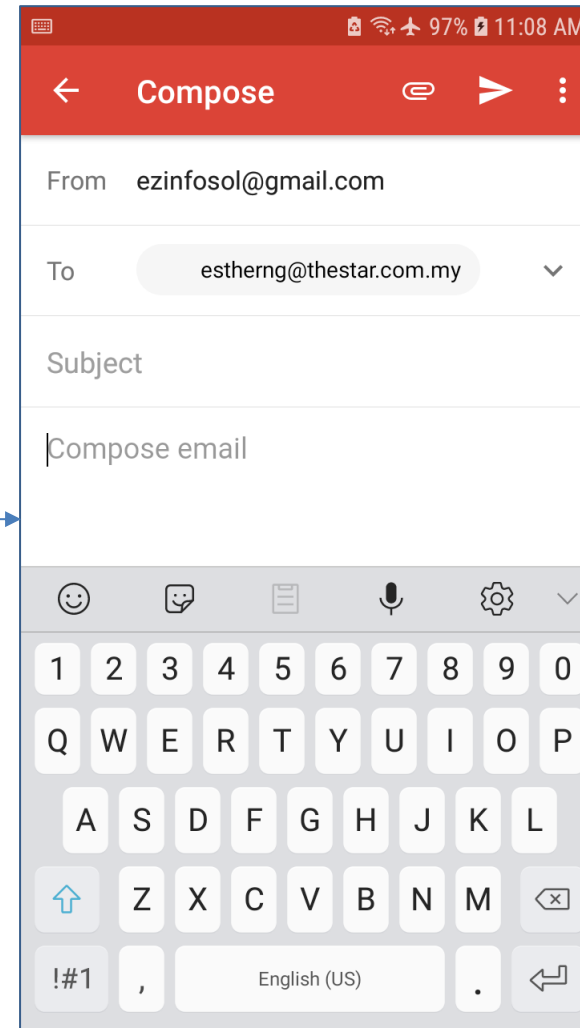


Browser (Contact)

Intent - Implicit



Browser (Contact)



Gmail (Compose)

Intent - Implicit

```
Intent emailIntent = Intent(Intent.ACTION_SEND)
// The intent does not have a URI, so declare the "text/plain" MIME type
emailIntent.setType(HTTP.PLAIN_TEXT_TYPE)

// recipients
emailIntent.putExtra(Intent.EXTRA_EMAIL, "jon@example.com")

emailIntent.putExtra(Intent.EXTRA_SUBJECT, "Email subject")
emailIntent.putExtra(Intent.EXTRA_TEXT, "Email message text")
emailIntent.putExtra(Intent.EXTRA_STREAM,
                    Uri.parse("content://path/to/email/attachment"))
```

Intent - Implicit

```
val callIntent: Intent = Uri.parse("tel:5551234").let { number ->
    Intent(Intent.ACTION_DIAL, number)
}
```

Or

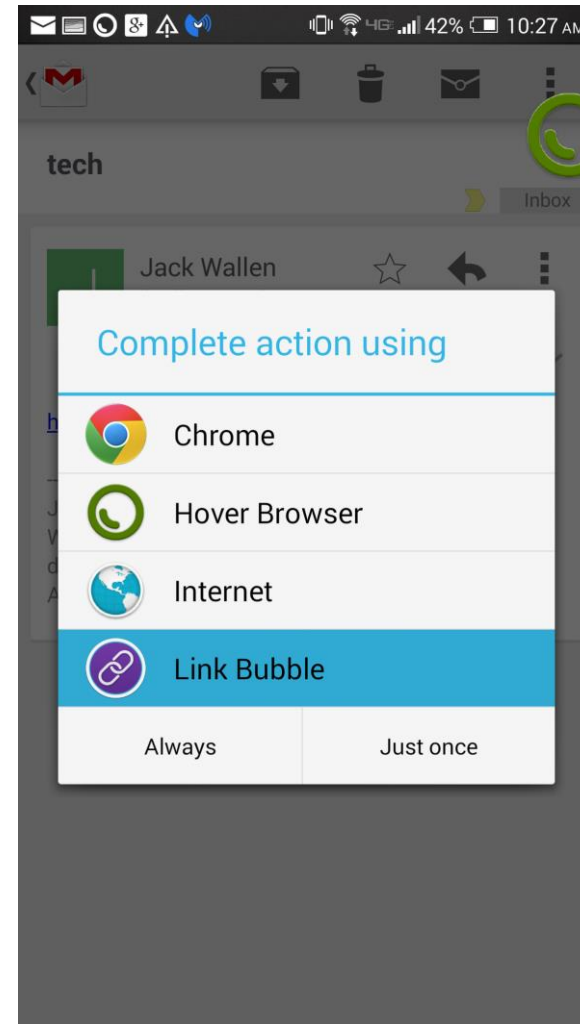
```
val callIntent = Intent(Intent.ACTION_DIAL)
callIntent.data = Uri.parse("tel:5551234")
```

// Map point based on address

```
val mapIntent: Intent = Uri.parse(
    "geo:0,0?q=1600+Amphitheatre+Parkway,+Mountain+View,+California"
).let { location ->
    // Or map point based on latitude/longitude
    // Uri location = Uri.parse("geo:37.422219,-122.08364?z=14") // z param is zoom level
    Intent(Intent.ACTION_VIEW, location)
}
```

Intent - Implicit

If multiple intent filters are compatible, the system displays a dialog so the user can pick which app to use



Intent - Implicit

It's possible that a user won't have *any* apps that handle an implicit intent

```
// Verify that the intent will resolve to an activity
if (sendIntent.resolveActivity(packageManager) != null) {
    startActivity(sendIntent)
} else
    // Inform user intent is unresolved
}
```

Intent - Implicit

For app targeting API Level 30 and above:

Declare your app's need for increased package visibility using the `<queries>` element in the manifest file:

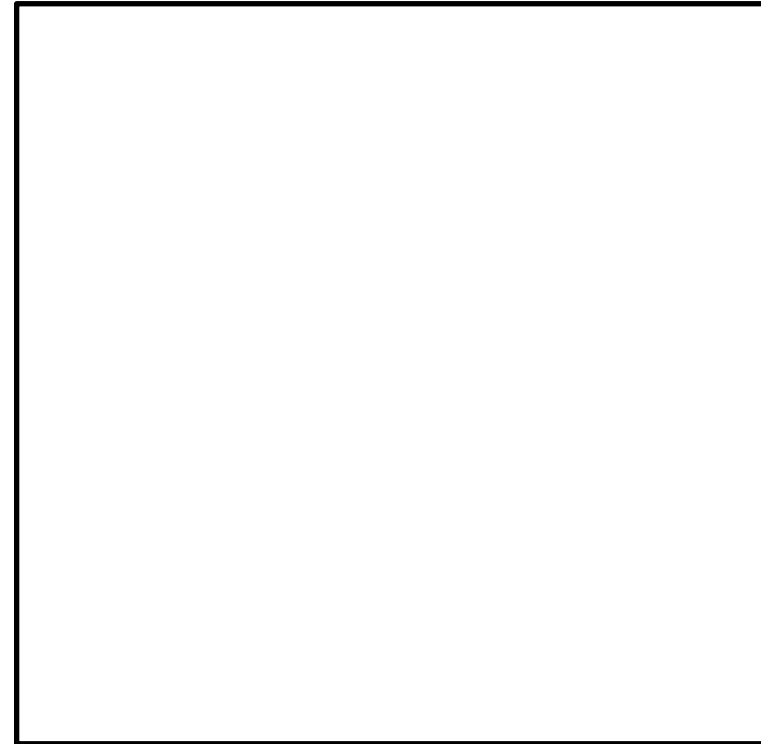
```
<queries>
  <!-- Browser -->
  <intent>
    <action android:name="android.intent.action.VIEW" />
    <data android:scheme="http" />
  </intent>

  <!-- Camera -->
  <intent>
    <action android:name="android.media.action.IMAGE_CAPTURE" />
  </intent>

  <!-- Gallery -->
  <intent>
    <action android:name="android.intent.action.GET_CONTENT" />
  </intent>
</queries>
```

Question?

1. What is the main difference between explicit and implicit intents?
2. Among explicit and implicit intents, which one is suitable for each of the following tasks:
 - a. To share a web site URL
 - b. To listen to an audio file downloaded from the Internet
 - c. To change an app sound profile to silent mode



Intent Filter

To advertise which implicit intents your app can receive using the `<intent-filter>` element

E.g. An activity declaration with an intent filter to receive an [ACTION_SEND](#) intent when the data type is text

```
<activity android:name="ShareActivity">
    <intent-filter>
        <action android:name="android.intent.action.SEND"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <data android:mimeType="text/plain"/>
    </intent-filter>
</activity>
```

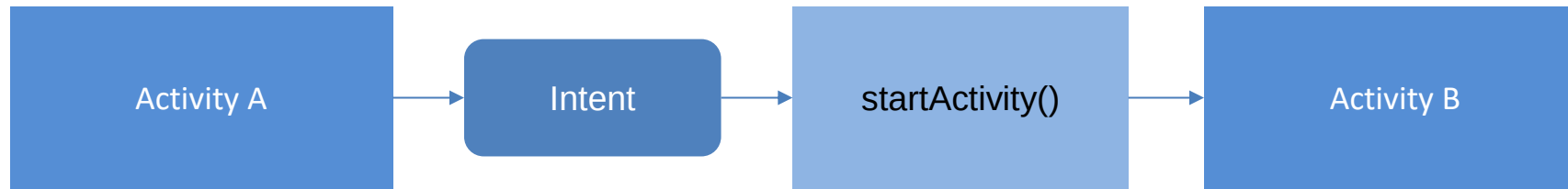
Getting a result from an Activity

You can start an Activity and receive a result back.

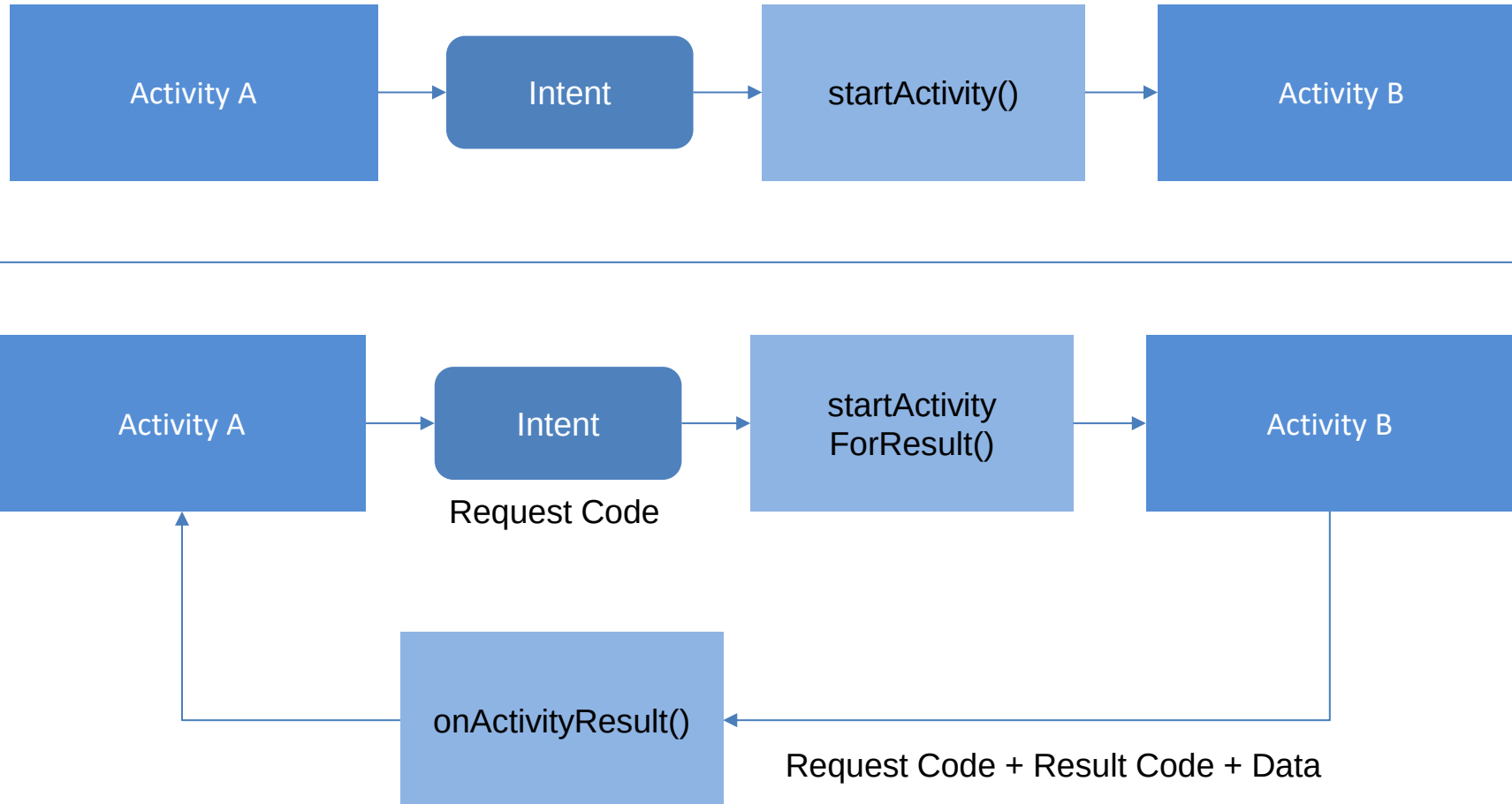
Two methods:

- [startActivityForResult\(\)](#) method (Deprecated): start the activity which will return a result
- [onActivityResult\(\)](#) callback method : to receive the result from the subsequent activity

Getting a result from an Activity



Getting a result from an Activity



```

const val PICK_CONTACT_REQUEST = 1 // The request code
...
private fun pickContact() {
    // Show user only contacts w/ phone numbers
    Intent(Intent.ACTION_PICK, Uri.parse("content://contacts"))
        .also { pickContactIntent -> pickContactIntent.type = Phone.CONTENT_TYPE

        startActivityForResult(pickContactIntent, PICK_CONTACT_REQUEST)
    }
}

...
override fun onActivityResult(requestCode: Int, resultCode: Int, data: Intent) {
    // Check which request we're responding to
    if (requestCode == PICK_CONTACT_REQUEST) {
        // Make sure the request was successful
        if (resultCode == Activity.RESULT_OK) {
            // The user picked a contact.
            // The Intent's data Uri identifies which contact was selected.

            // Do something with the contact here (bigger example below)
        }
    }
}

```

Activity Result API (New)

```
//In the module level
val getContent = registerForActivityResult(ActivityResultContracts.GetContent()) { uri: Uri? ->
    // Handle the returned Uri
    binding.apply {
        textViewPath.text = uri.toString()
        imageView.setImageURI(uri)
    }
}
```

```
getContent.launch("image/*")
```

<https://developer.android.com/training/basics/intents/result>

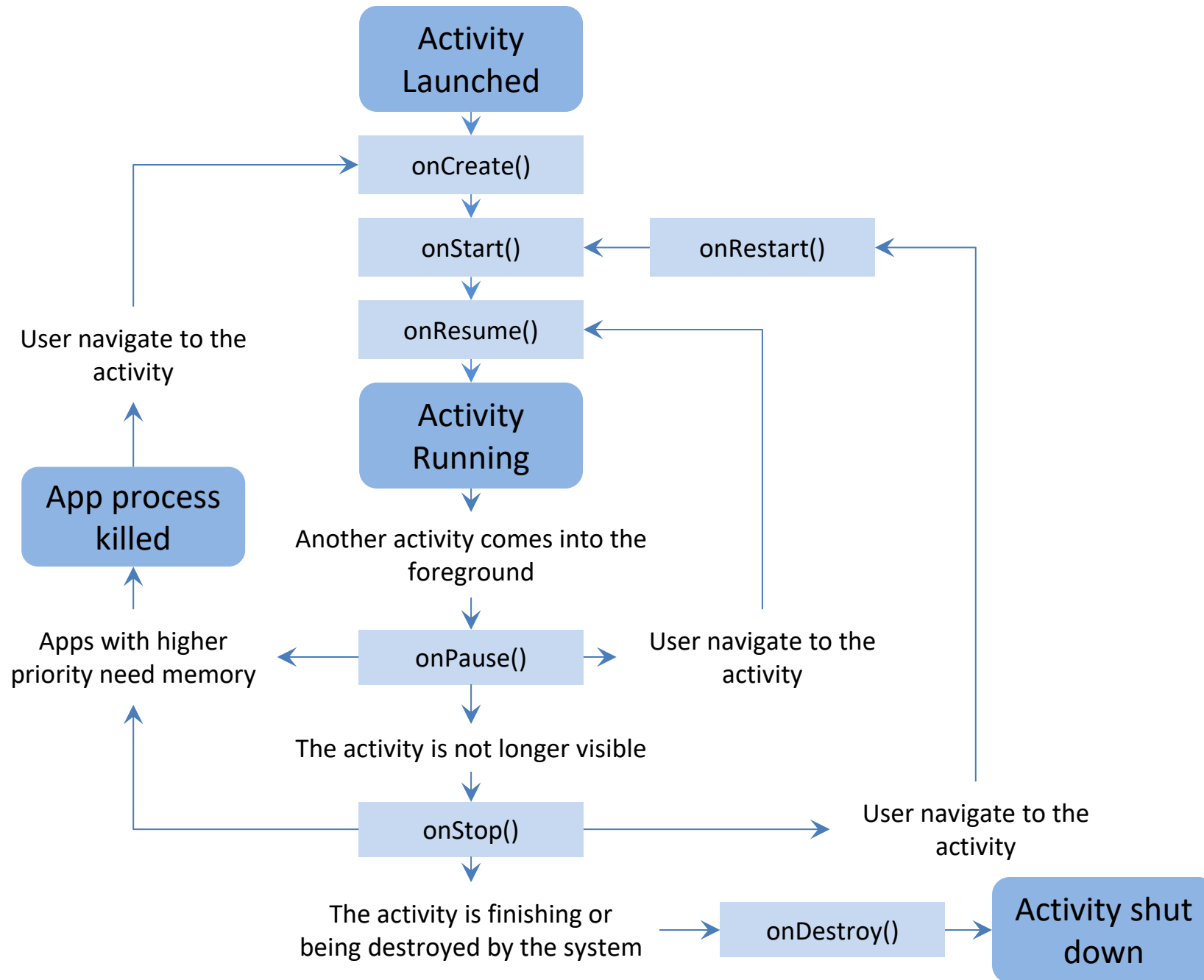
Activity Lifecycle

Activity instances of app transition through different states in their lifecycle

The Activity class provides callbacks that allow the Activity to know that a state has changed:

1. onCreate
2. onStart
3. onResume
4. onPause
5. onStop
6. onDestroy

Activity Lifecycle



Activity Lifecycle - States

State	Resumed 	Paused 	Stopped 	Destroyed 
Visible	Yes	Yes (Partially)	No	No
User Interaction	Yes	No	No	No
Code Execution	Yes	No	No	No
Instance State	Saved	Saved	Saved	Destroyed

Activity – 1. onCreate

It creates the Activity

Performs basic application startup logic that should happen only once for the entire life of the activity

E.g. Setup the app UI and instantiate some class-scope variables

- Initialize **Jetpack Compose UI** (setContent {})
- Set up **ViewModel**, LiveData, or state management
- Register **listeners or observers**
- Check **permissions** if needed

Activity – 2. onStart

- Makes the Activity visible to the user
- Activity enters the foregroundActivity becomes interactive
- Register **BroadcastReceivers** (if UI needs updates)
- Start animations or background services
- Load lightweight UI-related data
- Since Jetpack Compose doesn't have direct lifecycle methods,

```
@Composable
fun MyScreen() {
    LaunchedEffect(Unit) {
        Log.d("Lifecycle", "onStart Equivalent")
    }
}
```

Activity – 3. onResume

Activity comes to the foreground

Could be called several times:

- After onStart(): first call

- After onPause(): user return to the activity

Initialize components that you release during onPause(), and perform any other initializations that must occur each time the activity enters the Resumed state. E.g. starting a camera preview

Activity – 3. onResume

```
@Composable
fun MyScreen() {
    val lifecycleOwner = LocalLifecycleOwner.current

    LaunchedEffect(lifecycleOwner) {
        lifecycleOwner.lifecycle.addObserver(object : DefaultLifecycleObserver {
            override fun onResume(owner: LifecycleOwner) {
                Log.d("Lifecycle", "onResume - UI Fully Interactive")
            }
        })
    }
}
```

Activity – 4. onPause

User is leaving an activity. Does not always mean the activity is being destroyed

The Activity is no longer in the foreground but still visible in multi-window mode

Commit any changes that should be persisted beyond the current user session (save unsaved data)

Execution must very brief. Stop things that consume CPU, pause animations, media playback or real-time updates, and release foreground-only sensors

Activity – 5. onStop

An activity is no longer visible to the user

Releases almost all resources that aren't needed

Performs relatively CPU-intensive shutdown operations

E.g. save information to a database

Activity – 6. onDestroy

It is called because:

- the `finish()` is called or the system destroying the process containing the activity to save space
- an orientation change occurs (will be discussed later)

Releases all resources

The system may skip this callback

Activity - Configuration Change

Configuration Change:

- E.g. screen orientation, language, and input devices
- Current activity will be *destroyed*, going through the normal activity lifecycle process of [onPause\(\)](#), [onStop\(\)](#), and [onDestroy\(\)](#)

Jetpack Compose & Activity Lifecycle

- Jetpack Compose follows Android Activity Lifecycle, but it introduces Composables which have their own lifecycle that reacts to recompositions.
- Unlike traditional Views, Jetpack Compose does not have direct lifecycle callbacks instead LaunchedEffect is used to handle lifecycle events.

Jetpack Compose & Activity Lifecycle

```
@Composable
fun MyScreen() {
    val lifecycleOwner = LocalLifecycleOwner.current

    LaunchedEffect(lifecycleOwner) {
        lifecycleOwner.lifecycle.addObserver(object : DefaultLifecycleObserver {
            override fun onResume(owner: LifecycleOwner) {
                Log.d("Lifecycle", "Screen Resumed")
            }
            override fun onPause(owner: LifecycleOwner) {
                Log.d("Lifecycle", "Screen Paused")
            }
        })
    }

    Text("Hello Jetpack Compose!")
}
```

Jetpack Compose & Activity Lifecycle

- To perform lifecycle-dependant tasks(e.g Fetching data) use `rememberCoroutineScope()`

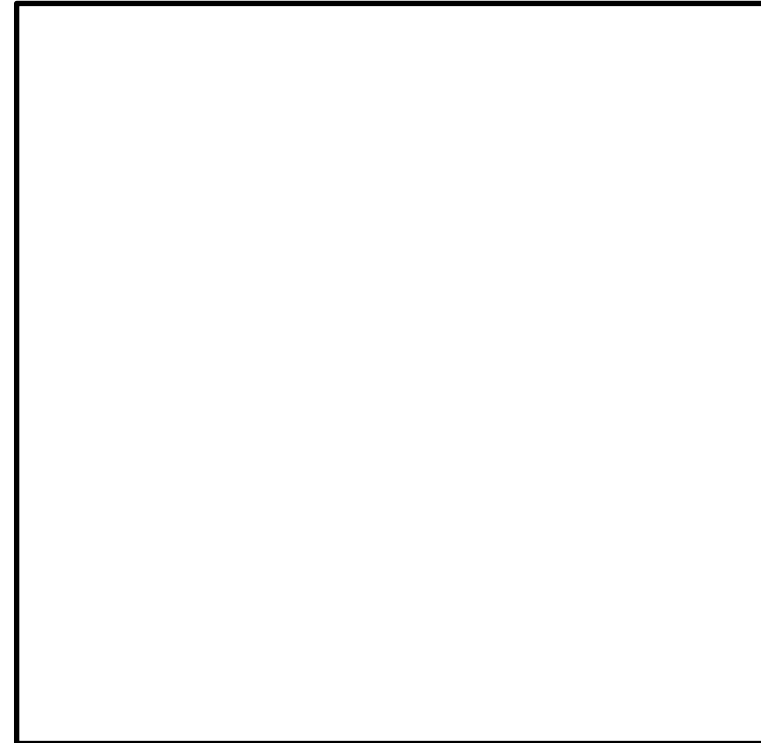
```
@Composable
fun FetchDataOnStart() {
    val coroutineScope = rememberCoroutineScope()

    LaunchedEffect(Unit) {
        coroutineScope.launch {
            delay(1000) // Simulate network call
            Log.d("Lifecycle", "Data Fetched")
        }
    }

    Text("Fetching Data...")
}
```

Question?

1. Why the onCreate and onPause callbacks are important to an Activity?
2. Describe a scenario where onPause and onStop would not be invoked.
3. Identify a suitable callback to implement the following tasks:
 - a. Playing background music
 - b. Save game level
 - c. Establish connection to a server
4. Identify two callbacks/methods that could be used to save data in a database when an Activity moves between states.



Saving and restoring activity state

Instance state = the saved data that the system uses to restore the previous state

It is a collection of key-value pairs stored in a [Bundle](#) object

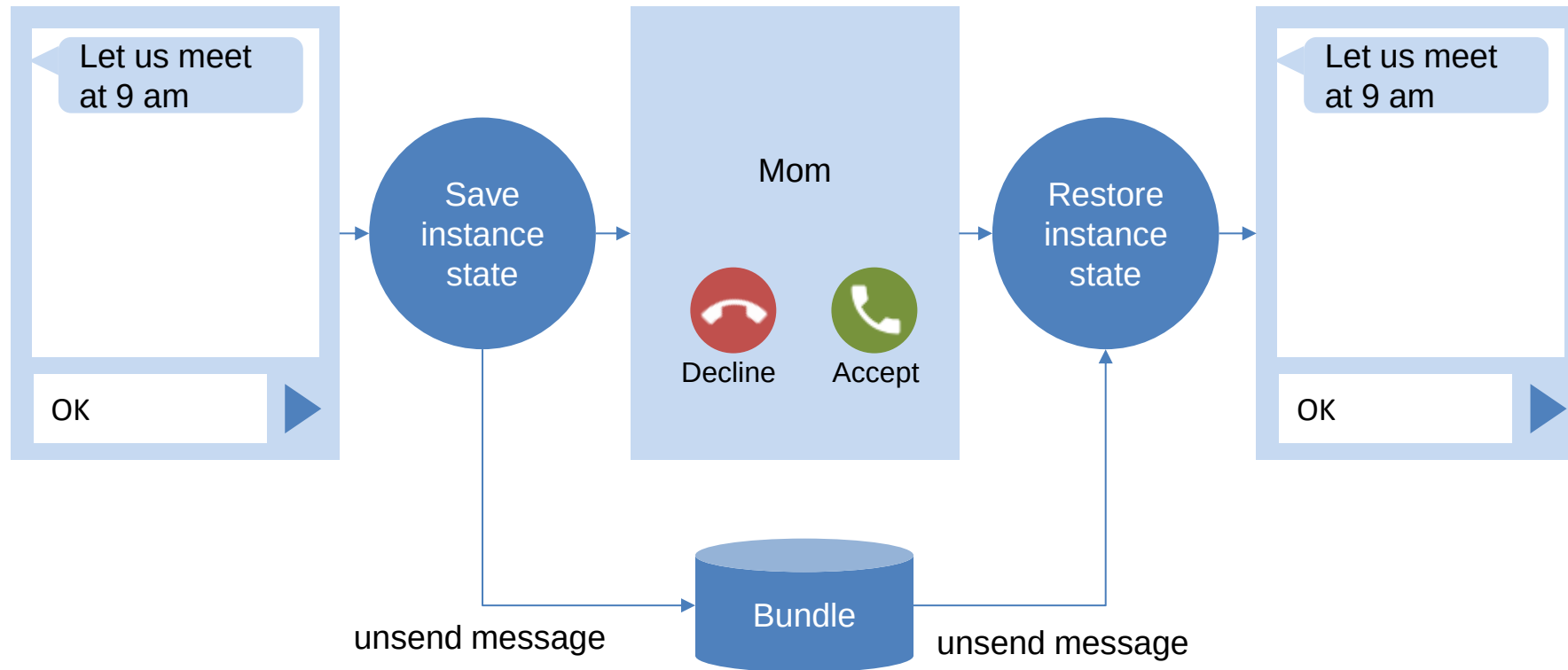
The system uses the [Bundle](#) instance state to save information about each UI component. E.g. value entered by user in a text field

Saving and restoring activity state

1. Composing a message

2. Showing phone app

3. Restoring instance state



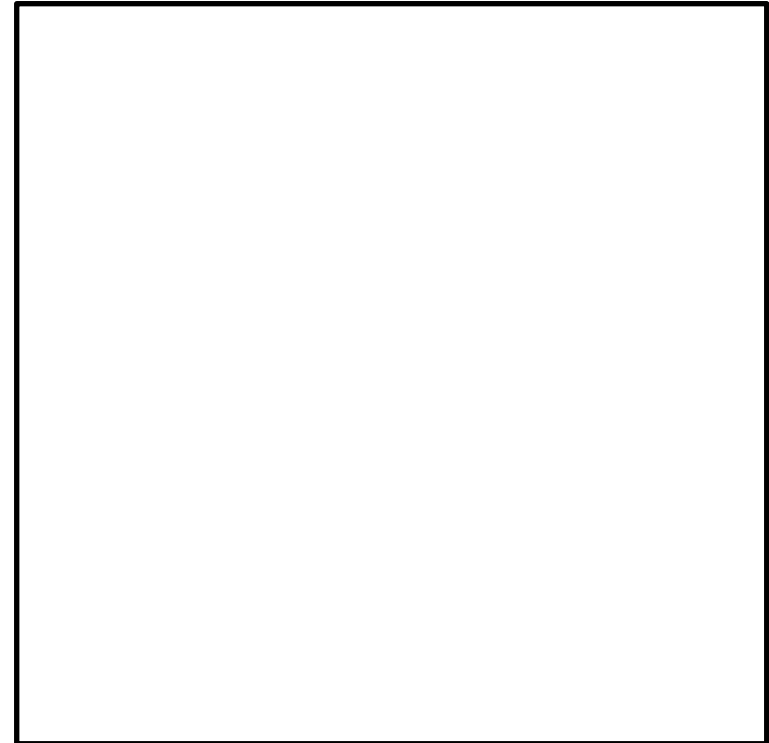
Saving and restoring activity state

```
lateinit var textViewMsg: TextView
...
override fun onRestoreInstanceState(savedInstanceState: Bundle?) {
    textViewMsg.text = savedInstanceState?.getString(MSG_VIEW_KEY)
}

override fun onSaveInstanceState(outState: Bundle?) {
    outState?.run {
        putString(MSG_VIEW_KEY, textViewMsg.text.toString())
    }
    // call superclass to save any view hierarchy
    super.onSaveInstanceState(outState)
}
```

Questions?

1. Multi-window and multitasking is a common feature of modern mobile devices. As a mobile app developer, explain how to handle the transition from one window/task to another to ensure important information is not lost.



Conclusion

Explain the concept of an Activity

Explain the Intent and Intent Filter

Explain the lifecycle of an Activity

Manage the Activity lifecycle